



MACQUARIE
University

MTRN6060 – Robotics and Automation

Semester: 2nd Semester 2025

Group 1 – 12 PM - Friday

Major Project Report

Project Title:

Robotic Automation for Automatic Transportation

Prepared by:

Kundan Singh Shekhawat

Email: kundansingh.shekhawat@students.mq.edu.au

Student ID: 48849855

Parth Pravin Shende

Email: parthpravin.shende@students.mq.edu.au

Student ID: 48654841

Gia Hung Tran

Email: giahung.tran@students.mq.edu.au

Student ID: 48574864

Date:

01/11/2025

TABLE OF CONTENTS

ABSTRACT	3
1. INTRODUCTION	4
2. DESIGN DETAILS	4
2.1 Subsystem 1 – Mobile Rover	4
2.2 Subsystem 2 – Boom Gate Station	5
2.3 System Workflow	5
3. MODELLING AND SIMULATION	7
3.1. SLAM Mapping & Waypoint Validation	7
3.2. Arm Calibration for Pick Up and Placement of Block	8
3.3. Boom-Gate Behaviour Modelling	9
4. TEST CASES AND RESULTS	9
5. SCHEMATICS, DRAWINGS, AND PHOTOS	10
5.1. Rover Hardware Layout	11
5.2. Boom Gate Circuit Schematic	11
6. CONCLUSION	11
6.1. Summarisation of work and results	11
6.2. Challenges, limitations and failures	12
6.3. Learning outcomes	13
7. REFERENCE	13

LIST OF FIGURES

Figure 1: Rover System Workflow	6
Figure 2: Boom Gate System workflow	7
Figure 3: Map Generated Using RVIZ SLAM	7
Figure 4: Rover Waypoint Locations	8
Figure 5: Arm Calibration Manual	8
Figure 6: Pick and Drop Calibration	9
Figure 7: Boom Gate Configuration	9
Figure 8: Rover Hardware Architecture	11
Figure 9: Boom Gate Circuit Diagram	11

LIST OF TABLES

Table 1: Rover Hardware Information (Hiwonder, 2025)	5
Table 3: Boom Gate Hardware Information	5

ABSTRACT

The project is designed to provide knowledge about the development of a robotic automation system. It focuses on integrating multiple systems, including ROS-based rover control, and promotes teamwork to meet the project timeline. The main objective was to design and develop an autonomous robotic solution that works in hazardous environments where it is unsafe or impractical for human workers. Additionally, the project can be used for repetitive tasks or to reduce labour tasks.

The whole system can be divided into two subsystems: a rover and a boom gate station. The rover needs to navigate the whole map or environment to pick up and deliver packages from fixed locations. The rover can avoid obstacles and create a 2D map using LiDAR. It can recognise colours using a 3D depth camera mounted on the robotic arm. The second subsystem is an Arduino-based boom gate that uses two ultrasonic sensors to sense the presence of the robot. The system uses a servo motor to open or close the boom gate.

The rover uses NVIDIA Jetson Orin Nano (8GB) and runs on ROS2, which can be configured using Python-based algorithms. It has a robotic arm with 6 DOF, with vision AI, and has SLAM mapping, multipoint navigation, object tracking, colour recognition, and edge/wall detection features.

As of now, our team has successfully recorded all the 2D way points in the map, which include the start point, pick up zone, boom gate location and drop zone. The most unique achievements required us to create LiDAR-based SLAM navigation and ROS 2-driven robot control manipulation to achieve a complete autonomous system.

1. INTRODUCTION

The project focuses on developing a fully automated robotic system that can move packages or materials. The goal is to create an understanding of the integration of rover, boom-gate hardware and software components using ROS to demonstrate navigation and communication capabilities between multiple systems. The rover can navigate the map using predefined points, pick colour-coded packages from a defined pickup point and finally drop them at the drop zone. Additionally, a second system, which uses an Arduino Nano as the main board, controls the boom that opens and closes by input from ultrasonic sensors.

The rover operates using a Python-based ROS 2 framework running on an NVIDIA Jetson Orin Nano. It is equipped with a LiDAR and a 3D depth camera to support Mapping (SLAM), recognising colours and packages. The system can perform navigation to multiple points using pre-recorded waypoints. It can also detect colours through image processing and use a 6 DOF robotic arm to complete pick-and-drop tasks. The boom gate is based on Arduino and uses ultrasonic sensors to detect the rover. It controls a servo motor for the gate mechanism. These two systems can imitate industry operations without any manual intervention.

The innovative aspect of this project is the integration of multipoint navigation and an automatic pick and place script, which requires a combination of a Lidar sensor, a robotic arm, along with STM32 and Jetson Orin Nano. We used multiple pickup points instead of only one pickup point to ensure all packages can be retrieved, even if they are scattered. We are using two diagonally positioned ultrasonic sensors on both sides of the boom gate to detect whether the rover is before or after the gate. We created a custom Python script that was developed to manage simultaneously. It can do navigation, pick-and-drop operations, and interact with the boom gate.

2. DESIGN DETAILS

In this section, we will talk about the functional design and operation of two subsystems, the rover and the Arduino-based boom gate. Both systems have their own set of programs and hardware integration to control sensors, servo motors and embedded logic.

2.1 Subsystem 1 – Mobile Rover

The rover uses the ROS2 Humble framework, where Jetson Orin Nano (8GB) is the primary controller and STM32 gets all the instructions through a USB serial interface from Jetson. Ros is responsible for navigation, current location, and environment with the help of LiDAR sensor, 3D depth camera, wheel encoder, and IMU. The navigation algorithm computes the optimal path to prerecorded waypoints. The rover loads the map saved using SLAM-based mapping and uses a colour detection algorithm for target detection([Hiwonder, 2024](#)).

Component	Specification / Function
Platform	JetRover M1 – Mecanum wheel chassis allowing omnidirectional movement
Motors	Hall Encoder DC motor
Main Controller	NVIDIA Jetson Orin Nano (8 GB) – ROS controller running ROS 2 Humble
STM32 Controller	Motion controller with in-built gyroscope and accelerometer
Operating System	Ubuntu with ROS 2 framework

Programming Language	Python
Software Interface	MobaXterm (used to send commands and monitor ROS topics)
Communication Method	Wi-Fi connection between the rover and the laptop for ROS command transfer
Servo Motors	Intelligent serial bus servo for robotic arm joints
Sensors	3D depth camera and LiDAR for SLAM-based mapping, colour detection, and obstacle avoidance
Control Method	CAN port communication and manual testing via remote controller

Table 1: Rover Hardware Information (Hiwonder, 2025)

The rover runs on Ubuntu, and we can control it with the help of a Windows system that has MobaXterm.

2.2 Subsystem 2 – Boom Gate Station

Component	Specification / Function
Microcontroller	Arduino Uno
Breadboard & Circuit	For sensor and servo interfacing
Sensors	Two ultrasonic sensors are placed diagonally to detect the rover's approach and departure.
Actuator	The servo motor controls the gate arm.
Communication	Independent system; monitors rover proximity and opens/closes the gate automatically.

Table 2: Boom Gate Hardware Information

The Arduino subsystem operates autonomously based on ultrasonic sensor inputs, ensuring synchronised interaction without manual intervention.

2.3 System Workflow

The system has multiple stages in one operating cycle. The rover's state transition is managed by ROS2 scripts and Arduino using sensors to control the boom gate. Our mission control boom gate script makes sure that all the tasks are working in coordination and running without interfering with each other.

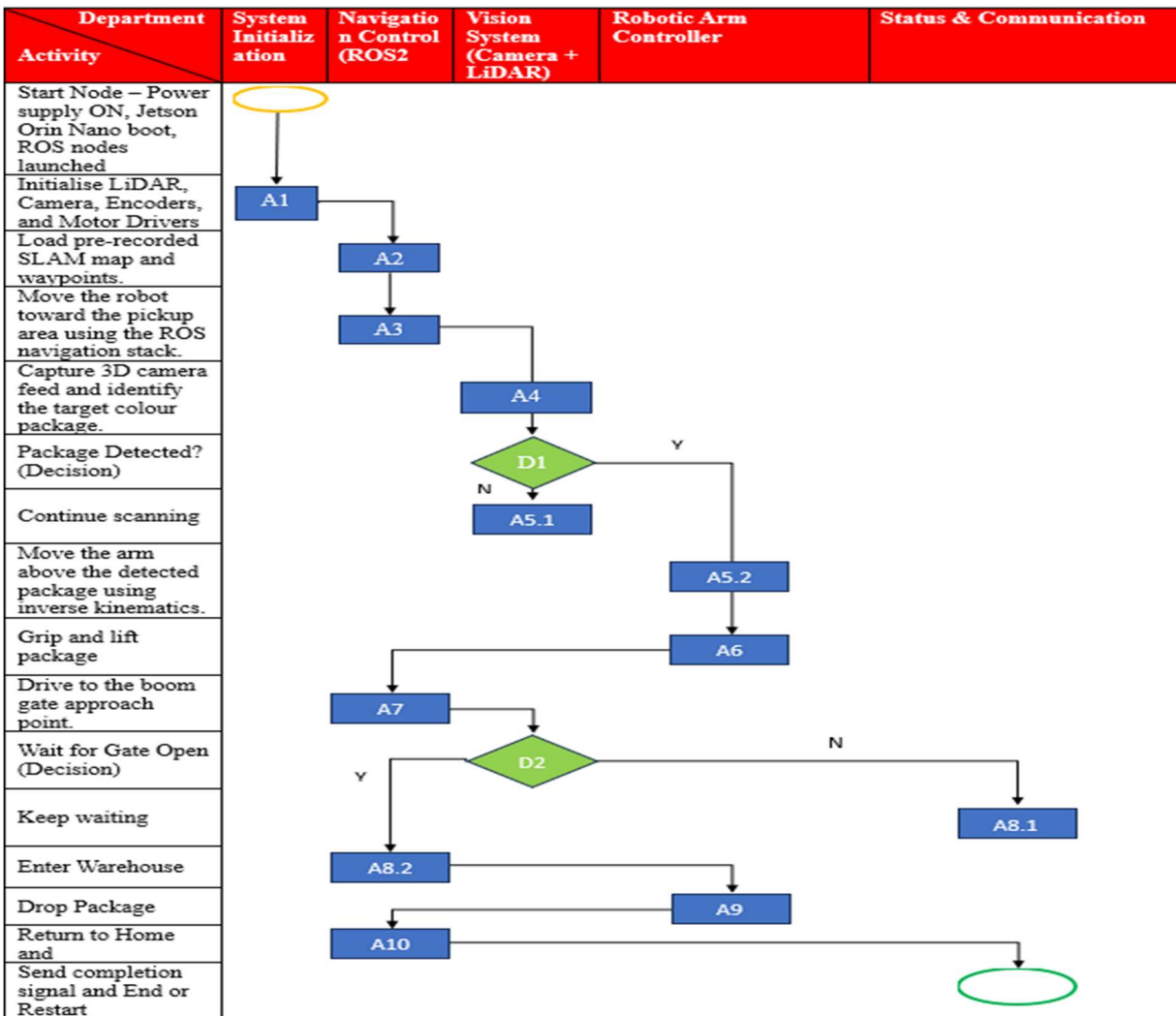


Figure 1: Rover System Workflow

Figure 2 explains the whole process of one operation cycle. It has included all the stages of the rover at each point. Before starting, we must send a few instructions:

- Kill all the current processes from the rover “sudo systemctl stop start_app_node.service”
- Load the map, start navigation & transport script using “ros2 launch example navigation_transport.launch.py map:=Sandbox_week10”
- Finally, we will start our own Python script “mission_control_GROUP1.py”

After this, the Robot will start the initialisation process by running path planning and navigation system, turning on the LiDAR scanner, and creating service clients for automatic pick up/place call. Once it is done, it can move towards the navigation waypoints and call pick-up/drop scripts at a pre-decided point in the mission control script. The rover will be able to know the location of the boom (before and after) so that it can start the LiDAR detection and wait for the boom gate to open. The rover will move if the gate is open; otherwise, it will wait for the boom gate to open.

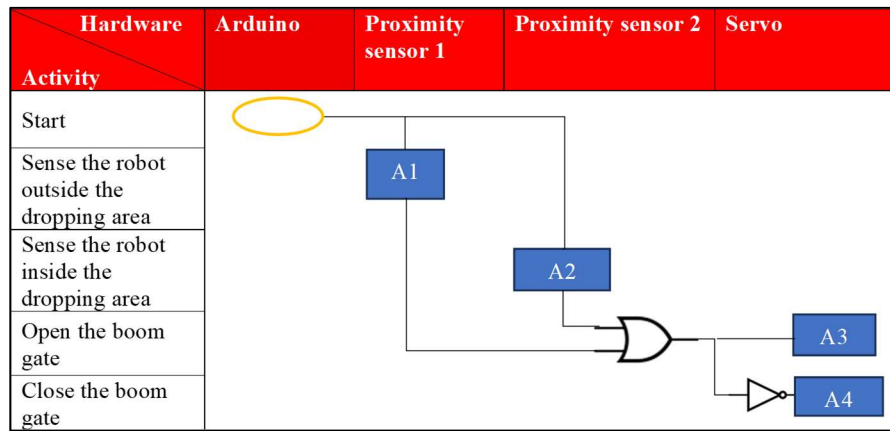


Figure 2: Boom Gate System workflow

Figure 2 explains the workflow of the boom gate. Both ultrasonic sensors are working throughout the whole cycle and continuously scanning the environment of the rover's arrival. When these sensors detect the rover before or after the gate, they send a signal to Arduino, and it instructs the servo to open the gate. The gate will remain in open position until it detects the rover on the other side. When the rover passes through the gate location, Arduino instructs the servo to close the gate.

3. MODELLING AND SIMULATION

3.1. SLAM Mapping & Waypoint Validation

Mapping was done (see figure 3) by manually driving the Jetrover robot to navigate around the sandbox whilst the LiDAR sensor continuously published distance measurements. The SLAM (Simultaneous Localisation and Mapping) algorithm, executed on the onboard controller, processed these measurements to build a virtual map of the sandbox environment.

RViz was used to visualise the LiDAR signals, monitor the generated map, and verify robot pose estimation with respect to the physical environment. This process was iterated until the complete map was navigated, and a 2D virtual map of the environment was developed.

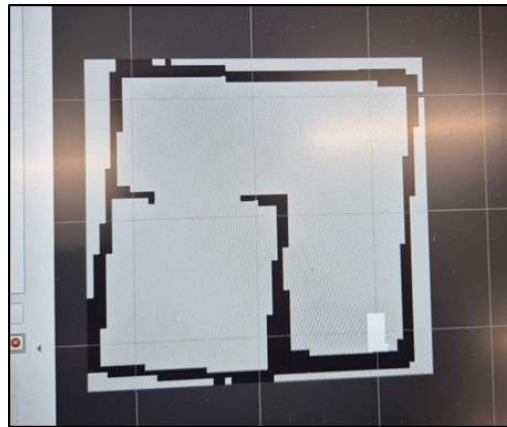


Figure 3: Map Generated Using RVIZ SLAM



Figure 4: Rover Waypoint Locations

Figure 4 shows the robot positions at the pickup locations on the map. The robot was manually navigated to each of the points to create an autonomous navigation path. Additional positions include:

- a) Home Position (Start Point)
- b) Pick Up Position
- c) Boom Gate Waiting Position
- d) Drop-off Position
- e) Boom Gate Exit Position

All these points were modelled as pose targets as (x, y, yaw angle) coordinates.

The navigation stack on the robot helps it navigate the points by avoiding the obstacles, which handles minor changes in the physical environment. The robot was manually navigated to each waypoint using the Nav2Goal command, and the entire path was later saved on the robot.

3.2. Arm Calibration for Pick Up and Placement of Block

The robotic arm was manually calibrated with the joint control interface provided by the manufacturer to allow picking up blocks of different sizes reliably. Each servo motor can be controlled directly by the interface by making steps in the position, and the arm reach, height, and gripper position can be finely adjusted.

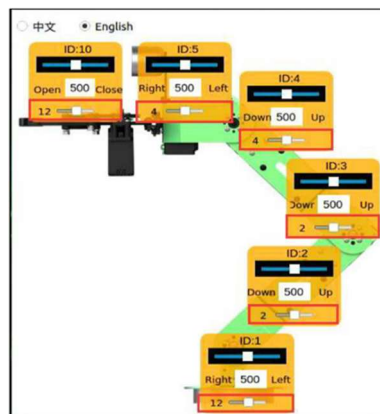


Figure 5: Arm Calibration Manual

Calibration Procedure:

Initial Positioning: The arm was initially positioned in a neutral home position to ensure that all the joints began at a familiar point.

Approach Alignment: The arm was moved with the help of ID:1 and ID:5 until the gripper was facing the centre of the target block.

Height & Reach Adjustment: The arm parts (ID:2, ID:3, ID:4) were moved one by one to find the right:

Pickup Calibration of Various Block Sizes: The final gripper approach height and closing distance were measured in each block size. This made sure that the gripper exerted adequate force to ensure that the block was firmly gripped without being deformed or moved.

Gripper actuation: The adjustments were made in ID:10 until the gripper was optimally closing to grip the block firmly.

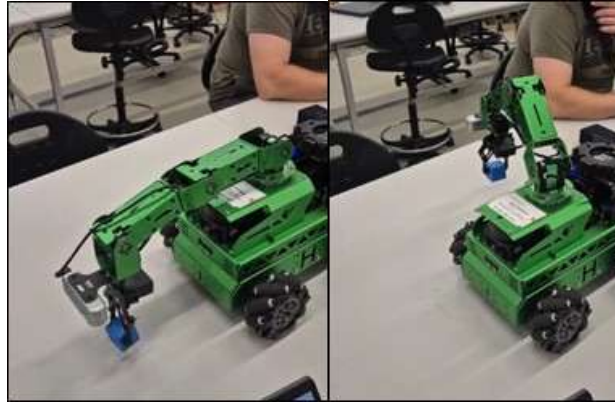


Figure 6: Pick and Drop Calibration

3.3. Boom-Gate Behaviour Modelling

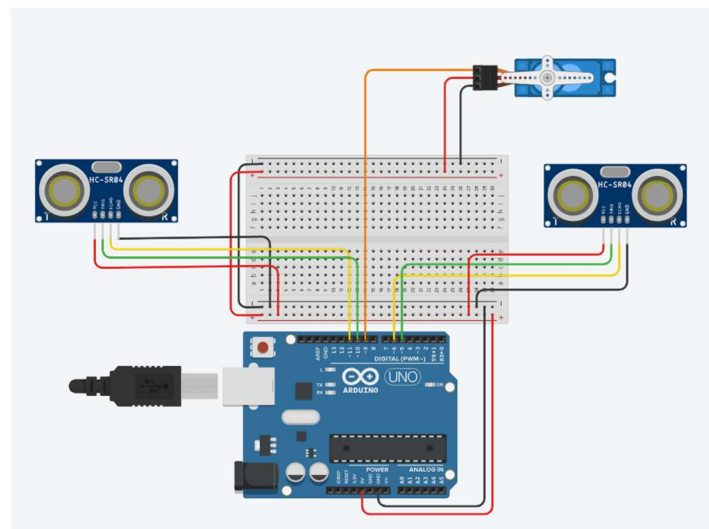


Figure 7: Boom Gate Configuration ([Tinkercad, 2025](#))

Our boom gate setup works with two ultrasonic sensors placed diagonally at entry and exit locations to detect the rover. We initially tried the Parallel sensor setup, where both sensors are facing the direction of the boom gate, but rover detection was too late, so we came up with an idea where, if they are placed diagonally, then they can detect the rover from an angle.

4. TEST CASES AND RESULTS

4.1. Navigation

Test case 1: Do we have enough waypoints?

We added 3 waypoints to pick up scattered blocks so rover can move to pick up location as soon as it detects a block and can skip remaining way point which are dedicated for pick

up purposes and move on to the before boom gate way point., but since we had very little time in the end to work on the actual script, which takes care of the navigation and automatic pick up we were not able to implement this strategy in our final demonstration.

Test case 2: Observe the rover behaviour at all points and modify it accordingly:

Our main test cases were about how the robot navigates through the poses we made and its initial position in the sandbox. We had to adjust the coordinates and the initial position to be too close to any wall, so as not to confuse the robot. Sometimes, if the goal is too close to a wall, the robot would get stuck between trying to avoid it and reaching the goal; other times, if the robot is too close to a wall, it would not notice it and couldn't figure out where it is on the map.

4.2. Pick Up and Drop

Test case 1: Is the rover able to pick up different sizes of blocks?

Our test cases were trying to pick up different block sizes with different orientations, and the main part to adjust was the gripper's width to make sure that it is wide enough to allow some room for error but still narrow enough to reliably grip the smaller blocks.

Test case 2: Can the rover arm recognise a blue colour block among other colours?

This was one of the test robots that failed multiple times, as we could see the robot could pick a block when it was clearly away from other blocks. We also found out that when there are multiple blocks of the same colour, it takes a lot of time to decide which one to pick first.

4.3. Boom Gate

Test case 1: Can the boom gate detect the rover on time and react appropriately?

We tested the boom gate with the most optimal setup of ultrasonic sensors and finally decided on a diagonal setup, which can detect the rover from at least a 6-inch away from the boom gate to ensure the gate opening before the rover can reach the gate.

Test case 2: Do we need a delay in closing the gate so that it doesn't hit the rover when it is trying to pass through the gate?

We also added a delay in closing the boom gate so that when there is no rover, it does close instantly if, for some reason rover is in a blind zone of both the ultrasonic sensors.

5. SCHEMATICS, DRAWINGS, AND PHOTOS

In this section, we will document the hardware layout and wiring diagram of the rover and the boom gate, showing how mainboards, controllers, servos and other components are connected to each other.

5.1. Rover Hardware Layout

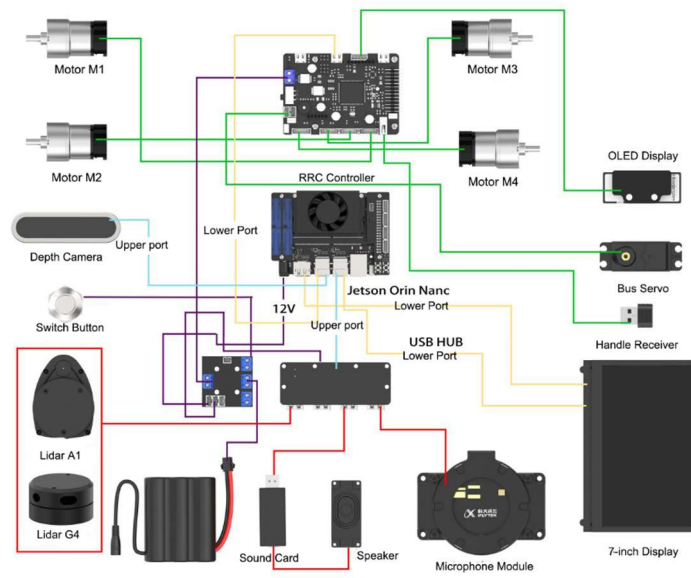


Figure 8: Rover Hardware Architecture

5.2. Boom Gate Circuit Schematic

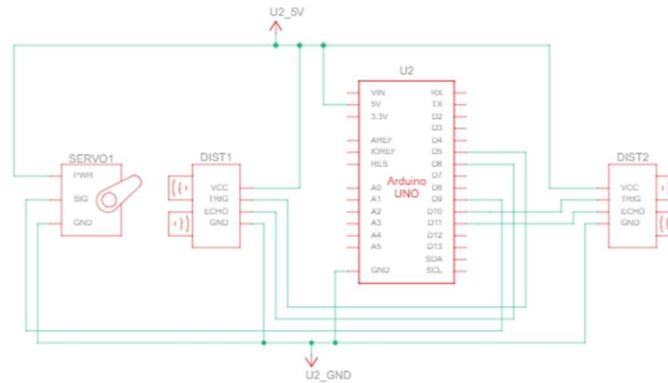


Figure 9: Boom Gate Circuit Diagram ([Tinkercad, 2025](#))

6. CONCLUSION

6.1. Summarisation of work and results

In this project, we worked on controlling the robot manually and automatically, as well as making a boom gate system.

Manual Control of the rover:

- We controlled its movement through the keyboard control program and the joystick.
- The manual movement control was used in conjunction with SLAM using the LiDAR to create a 2D map of the robot's working area.
- The robot can be moved by creating a goal in RViz rather than through buttons or a joystick.

Robotic Arm Operation:

- We manually jogged its joints through the arm software
- We also moved it through poses that are adjusted by hand.

- The arm program was used to calibrate the arm's joints until the automatic pick program runs reliably with a certain block size.

Automatic Control:

- We successfully run an automatic loop where the robot moves from the starting point to the pickup point and waits for 30 seconds for the robot to detect and try to pick up a block.
- Next, it will go to the boom gate and wait for the boom gate to open before coming into the dropping area, drop the block and turn back around and wait in front of the boom gate for it to open before going back to its home position.

Boom gate system:

- We made it with an Arduino, 2 proximity sensors that point outwards diagonally from the boom gate.
- A servo motor mounted on a cardboard box with a gate made of wooden sticks attached to it, and sits at the LiDAR's height in its closed position. It functions as expected.

6.2. Challenges, limitations and lessons learned

The two main technical challenges were calibrating both the joints and the camera and writing a script to fully automate the robot's operation. In terms of calibration, we had to calibrate the joints by eye first, then test them each time using the automatic pick program, and it would fail if either the arm or the camera was not calibrated properly; this process requires a lot of trial and error. For the programming part, we were not familiar with the libraries and general operations of this robot, making the debugging process rather laborious, even if the idea or algorithm behind the program is quite simple.

Before the day of the demonstration:

- The biggest challenge in this project was how unresponsive the robot was. It was clear that the robot was not industrial grade from the get-go, but it was surprisingly unresponsive.
- Each step of the way, like launching the navigation transport program, waiting for the map and camera input to load up, estimating the pose, etc., takes a lot of time, and any of the steps can freeze up the system even when doing basic tasks even after having run the two force-stopping all processes command (with `ign` and `ros`) and `stop_start_app_node.service`.
- The constantly disconnecting Wi-fi certainly made things way worse than they already were. Every session we worked with the robot, at least half the time was wasted doing the same basic things until the robot started working as it should.
- Each group had to deal with those challenges and then wait in line to get into a single sandbox to test things out with no support for simulation, which leaves us with a very rushed schedule, even with long sessions.
- Of course, we debugged our code ourselves as thoroughly as we could at home, but in the short periods of time when we had our robot in the sandbox and it actually functioned somewhat properly, we had to prioritize trying to get it to achieve the objective as quickly as we could and ignoring other parts and even then, we were still short on time.

- We managed to test the automated process with the desired outcome right before the demonstration.

During the demonstration:

- The robot failed to pick up the block even though we had already calibrated it for that block, and it had picked it up before.
 - One of the causes might be that there were some dust and grime on the ground and the robot's wheels, causing the slide on the ground and creating a kind of mismatch between the feedforward and feedback data of its position.
- The rover turned to the wall before going out of the dropping area, and the reason for that seems to be that it was looking for the shortest path and was trying to see if the wall was still there before finding out that it had to go through the boom gate.
 - This problem can be fixed by adding more middle points, which will allow us to fine-tune the predefined path, leaving less work for the path-finding algorithm.
- It also tried to pick up a block once while on its way back to the starting position. Instead of reaching the position first, this can be fixed by starting the navigation transport part of the process after reaching the starting point and stopping it after dropping every cycle.
- We also did not have enough time for the traffic light part; to apply it, we would need to use the colour sorting program without the picking/placing part and read the status of that process and proceed accordingly (ending the process or running another loop).

On a side note, before the break, one of the links in the arm was also crooked, making the camera position also crooked, giving the system a false perception of a block's position with respect to the end effector, making the automatic pick program fail regardless of the joints' calibration. The 3D mapping program also made the robot crash out without fail.

6.3. Learning outcomes

In this project, we learned to interact with a robot using ROS and MobaXterm which includes: Jogging the arm joints, calibrating the arm joints, moving the arm into a pose adjusted by hand, calibrating the camera, manually moving the robot around with the keyboard or the joystick, sending and reading variables between nodes, mapping an area and moving the robot in that area in RViz, saving the robot's pose within a map, navigating a map automatically with the robot, picking a block with the robot automatically when it detects it with the camera's input.

We also controlled a motor using an Arduino with inputs from proximity sensors, but that was a very simple embedded system any mechatronics student should be expected to figure out rather than learn from any specific project.

7. REFERENCE

- Hiwonder. (2025). *JetRover M1: ROS-based intelligent robot car with Jetson Orin Nano*. Hiwonder. <https://www.hiwonder.com/products/jetrover?variant=41198996717655>
- Hiwonder. (2024). JetRover quick start guide: Development environment setup and configuration. Hiwonder Documentation. https://docs.hiwonder.com/projects/JetRover/en/jetson-orin-nano/docs/1.Quick_Start_Guide.html#development-environment-setup-and-configuration
- Tinkercad (2025). *Tinkercad circuit simulation project*. Tinkercad. <https://www.tinkercad.com/things/gBH3a2SnfaG/editel>